

Engine de Busca Distribuída

Arquitetura, Recuperação de Informação e Escalabilidade

Projeto backend baseado em Java 25, Spring Boot 4, microserviços,
Recuperação de Informação, processamento assíncrono, indexação
distribuída e escalabilidade horizontal.

Documento de planejamento técnico e arquitetura

20 de setembro de 2026

Sumário

1	Visão geral do projeto	6
2	Objetivos	7
2.1	Objetivo principal	7
2.2	Objetivos acadêmicos de Recuperação de Informação	7
3	Princípios arquiteturais	8
3.1	Separação entre ingestão e consulta	8
4	Stack recomendada	9
5	Requisitos funcionais	10
5.1	RF01 — Cadastro de seeds	10
5.2	RF02 — Descoberta automática de páginas	11
5.3	RF03 — Canonicalização de URLs	11
5.4	RF04 — Controle de duplicidade	11
5.5	RF05 — Respeito ao robots.txt	11
5.6	RF06 — Controle de crawling por domínio	11
5.7	RF07 — Armazenamento do conteúdo coletado	11
5.8	RF08 — Extração de conteúdo	12
5.9	RF09 — Indexação	12
5.10	RF10 — Pesquisa textual	12
5.11	RF11 — Paginação	12
5.12	RF12 — Ranking	12
5.13	RF13 — Snippets	12
5.14	RF14 — Autocomplete	13
5.15	RF15 — Sugestão ortográfica	13
5.16	RF16 — Reindexação	13
5.17	RF17 — Análise de links	13
6	Requisitos não funcionais	13
6.1	Escalabilidade horizontal	13
6.2	Disponibilidade	13
6.3	Idempotência	13

6.4	Baixa latência	14
6.5	Observabilidade	14
6.6	Evolução independente	14
6.7	Tolerância a falhas	14
7	Conceitos de Recuperação de Informação aplicados	14
7.1	Documento	14
7.2	Tokenização	15
7.3	Normalização	15
7.4	Índice invertido	15
7.5	BM25	16
8	Arquitetura geral	17
9	Microserviços	18
9.1	API Gateway	18
9.2	Crawl Coordinator	19
9.3	Crawler Worker	19
9.4	Document Processor	20
9.5	Indexing Service	20
9.6	Search Service	21
9.7	Link Analysis Service	21
9.8	Query Intelligence Service	22
10	Fluxo de crawling e indexação	23
11	Fluxo de pesquisa	24
12	Estratégia de armazenamento	25
12.1	PostgreSQL	25
12.2	OpenSearch	25
12.3	Redis	26
12.4	Object Storage	26
13	Mensageria e eventos	26
13.1	Garantia de processamento	27
13.2	Schema Registry	27

14 Ranking e Recuperação de Informação	27
14.1 Ranking inicial	28
14.2 Segunda evolução	28
14.3 Terceira evolução	28
14.4 Quarta evolução	28
14.5 Quinta evolução	28
15 Diagramas de classes	29
15.1 Crawl Coordinator	29
15.2 Crawler Worker	29
15.3 Document Processor	29
15.4 Search Service	30
16 Arquitetura interna dos serviços	30
17 APIs principais	31
17.1 Search	31
17.2 Autocomplete	32
17.3 Administração de crawling	32
17.4 Consulta de execução	32
17.5 Recrawling	32
17.6 Reindexação	32
18 Escalabilidade e Kubernetes	32
18.1 Serviços stateless	33
18.2 Escalabilidade independente	33
18.3 HPA	33
18.4 KEDA	33
18.5 Service Discovery	34
18.6 Configuração	34
19 Resiliência e consistência	34
19.1 Retry	34
19.2 Circuit Breaker	35
19.3 Dead Letter Queue	35
19.4 Transactional Outbox	35

20 Segurança	35
20.1 SSRF	35
20.2 Restrições HTTP	36
20.3 API administrativa	36
20.4 API pública	36
21 Observabilidade	36
21.1 Métricas do Search Service	36
21.2 Métricas do Crawler	37
21.3 Métricas Kafka	37
21.4 Métricas do Indexer	37
21.5 Ferramentas	37
22 Testes e qualidade	37
22.1 Testes unitários	37
22.2 Testes de integração	38
22.3 Testes arquiteturais	38
22.4 Contratos	38
22.5 Performance	38
23 Avaliação de Recuperação de Informação	39
23.1 Precision@K	39
23.2 Recall@K	39
23.3 MRR	39
23.4 nDCG@K	39
23.5 Experimentos	40
24 Estratégia de índices	40
25 Organização do repositório	40
26 Compartilhamento de código	41
27 Estratégia database-per-service	42
28 Arquitetura de deploy	43
29 Roadmap de desenvolvimento	43

29.1 Fase 1 — Recuperação de Informação básica	43
29.2 Fase 2 — Crawler	44
29.3 Fase 3 — Arquitetura orientada a eventos	44
29.4 Fase 4 — Crawling distribuído	44
29.5 Fase 5 — Qualidade de ranking	45
29.6 Fase 6 — Kubernetes	45
29.7 Fase 7 — Observabilidade	45
29.8 Fase 8 — Busca semântica	46
30 O que implementar manualmente e o que utilizar pronto	46
30.1 Vale implementar ou experimentar	46
30.2 Não vale reinventar inicialmente	47
31 Decisões arquiteturais principais	47
32 Stack final recomendada	48
33 Princípio central da arquitetura	49

1. Visão geral do projeto

O objetivo é desenvolver uma **engine de busca distribuída**, inspirada conceitualmente em mecanismos como Google e Bing, porém em uma escala adequada para estudo, experimentação e aplicação prática de conceitos de Engenharia de Software e Recuperação de Informação.

O sistema deverá ser capaz de:

- descobrir páginas;
- visitar páginas automaticamente;
- extrair informações relevantes;
- identificar links;
- construir um corpus de documentos;
- criar índices;
- processar consultas;
- recuperar documentos;
- ranquear resultados;
- avaliar a qualidade das respostas;
- escalar horizontalmente os componentes mais utilizados.

O projeto também deverá servir como exercício de:

- arquitetura de software;
- sistemas distribuídos;
- microserviços;
- mensageria;
- observabilidade;
- tolerância a falhas;
- escalabilidade;
- containerização;
- CI/CD;
- boas práticas de desenvolvimento.

Ponto importante

Não é necessário reproduzir toda a infraestrutura interna do Google. O objetivo é reproduzir os principais problemas arquiteturais de uma engine de busca e aplicar soluções modernas e bem fundamentadas para resolvê-los.

2. Objetivos

2.1 Objetivo principal

A plataforma deverá suportar o fluxo conceitual:

```
Internet
  |
  v
Descoberta de URLs
  |
  v
Crawling
  |
  v
Extração
  |
  v
Processamento
  |
  v
Indexação
  |
  v
Recuperação
  |
  v
Ranking
  |
  v
Resultado da busca
```

Um usuário poderá pesquisar, por exemplo:

```
spring boot microservices
```

e receber documentos ordenados de acordo com sua relevância.

2.2 Objetivos acadêmicos de Recuperação de Informação

O sistema deverá permitir estudar e experimentar conceitos como:

- índice invertido;
- vocabulário;
- frequência de termos;
- TF;
- IDF;

- TF-IDF;
- modelo booleano;
- modelo vetorial;
- similaridade de cosseno;
- BM25;
- precisão;
- revocação;
- Precision@K;
- Recall@K;
- MAP;
- MRR;
- nDCG;
- stemming;
- remoção de stopwords;
- tokenização;
- normalização;
- expansão de consultas;
- análise de links;
- PageRank;
- busca semântica;
- busca híbrida.

Nem todos esses recursos precisam estar presentes na primeira versão.

3. Princípios arquiteturais

3.1 Separação entre ingestão e consulta

A arquitetura deverá separar o **pipeline de ingestão/indexação** do **pipeline de consulta**.

Pipeline de ingestão:

```
Crawler
  |
  v
Processamento
  |
  v
Indexacao
```

Esse fluxo é pesado, assíncrono e orientado a eventos.

Pipeline de consulta:

```
Usuario
  |
  v
API
  |
  v
Query Processing
  |
  v
Recuperacao
  |
  v
Ranking
  |
  v
Resultado
```

Esse fluxo é sensível à latência.

Decisão arquitetural

Crawling, parsing, processamento e reindexações nunca deverão bloquear a API de busca. Os dois pipelines devem poder escalar independentemente.

Essa separação permite, por exemplo:

```
50 instancias de crawler
10 processadores de documentos
8 indexadores
20 instancias de busca
```

sem que esses números precisem estar diretamente relacionados.

4. Stack recomendada

A stack base proposta é:

Área	Tecnologia sugerida
------	---------------------

Linguagem	Java 25
Framework	Spring Boot 4
Ecosistema cloud	Spring Cloud
Build	Maven
Banco relacional	PostgreSQL
Índice de busca	OpenSearch
Cache	Redis
Mensageria	Apache Kafka
Object Storage	S3-compatible storage
Containers	Docker
Orquestração	Kubernetes
API	REST
Contratos HTTP	OpenAPI
Contratos de eventos	Apache Avro
Schema Registry	Apicurio Registry
Métricas	Prometheus
Dashboards	Grafana
Tracing	OpenTelemetry
Logs	Loki
Migração SQL	Flyway
Testes de infraestrutura	Testcontainers
CI	GitHub Actions
CD	Argo CD, em uma etapa posterior

Boa prática

Antes da implementação, validar a matriz de compatibilidade entre Spring Boot 4, Spring Cloud e JDK 25 para a versão exata escolhida no início do projeto.

5. Requisitos funcionais

5.1 RF01 — Cadastro de seeds

O administrador poderá informar URLs que servirão como ponto inicial para o crawler.

```
https://spring.io
https://docs.oracle.com
https://kubernetes.io
```

5.2 RF02 — Descoberta automática de páginas

O crawler deverá extrair links das páginas visitadas e encaminhá-los para análise e possível inserção no URL Frontier.

5.3 RF03 — Canonicalização de URLs

URLs equivalentes deverão ser identificadas sempre que possível.

Exemplo:

```
https://site.com/  
https://site.com
```

não deverão gerar documentos diferentes sem necessidade.

5.4 RF04 — Controle de duplicidade

O sistema deverá impedir processamento desnecessário de:

- mesma URL;
- mesma URL canonicalizada;
- documentos com conteúdo idêntico ou altamente semelhante.

Hashes de conteúdo poderão ser utilizados como parte da estratégia de deduplicação.

5.5 RF05 — Respeito ao robots.txt

Antes de visitar um domínio, o crawler deverá considerar as regras publicadas em `robots.txt`.

5.6 RF06 — Controle de crawling por domínio

O sistema deverá aplicar:

- limite de concorrência;
- rate limiting;
- intervalo mínimo entre requisições;
- backoff após falhas;
- políticas específicas por domínio.

5.7 RF07 — Armazenamento do conteúdo coletado

HTML e outros conteúdos brutos deverão poder ser armazenados antes do processamento.

5.8 RF08 — Extração de conteúdo

Deverão ser extraídos, no mínimo:

- URL;
- URL canonical;
- título;
- descrição;
- texto;
- headings;
- idioma;
- links;
- data da coleta.

5.9 RF09 — Indexação

Os documentos processados deverão ser inseridos no índice utilizado pela engine.

5.10 RF10 — Pesquisa textual

Exemplo:

```
GET /api/v1/search?q=recuperacao+informacao
```

5.11 RF11 — Paginação

A API deverá aceitar paginação convencional em versões iniciais:

```
page  
size
```

Para grandes volumes, deverá evoluir para cursor ou *search-after*.

5.12 RF12 — Ranking

A primeira implementação de produção deverá utilizar BM25.

5.13 RF13 — Snippets

Cada resultado deverá poder apresentar um trecho contextual do documento.

5.14 RF14 — Autocomplete

Exemplo futuro:

```
spr
|
+--> spring
+--> spring boot
+--> spring security
```

5.15 RF15 — Sugestão ortográfica

Exemplo:

```
Consulta: sprng boot

Sugestao:
spring boot
```

5.16 RF16 — Reindexação

O sistema deverá permitir recriar índices sem interromper as consultas.

5.17 RF17 — Análise de links

O sistema deverá registrar relações entre documentos para possibilitar cálculo posterior de autoridade e PageRank.

6. Requisitos não funcionais

6.1 Escalabilidade horizontal

Os principais serviços deverão poder operar com múltiplas instâncias.

6.2 Disponibilidade

Falhas no pipeline de crawling não deverão indisponibilizar a API de busca.

6.3 Idempotência

O mesmo evento processado mais de uma vez não deverá produzir inconsistências.

6.4 Baixa latência

Como metas iniciais, sujeitas a validação por benchmark:

```
p50 < 100 ms
p95 < 300 ms
p99 < 1 s
```

6.5 Observabilidade

Todos os serviços deverão produzir:

- logs;
- métricas;
- traces;
- health checks.

6.6 Evolução independente

Um crawler deverá poder ser atualizado sem exigir atualização simultânea do Search Service.

6.7 Tolerância a falhas

Falhas temporárias deverão utilizar retry com backoff e jitter.

Falhas permanentes deverão poder ser enviadas para Dead Letter Queue.

7. Conceitos de Recuperação de Informação aplicados

7.1 Documento

Um documento pesquisável poderá possuir:

```
Document
|- id
|- url
|- canonicalUrl
|- title
|- description
|- content
|- language
|- links
|- crawledAt
|- contentHash
'- rankingSignals
```

7.2 Tokenização

Exemplo:

```
Entrada:
Spring Boot facilita aplicacoes Java

Tokens:
spring
boot
facilita
aplicacoes
java
```

7.3 Normalização

Possíveis etapas:

- lowercase;
- tratamento de acentos;
- stemming;
- remoção de stopwords;
- regras específicas por idioma.

7.4 Índice invertido

Para:

```
D1: spring boot java
D2: spring security java
D3: angular typescript
```

conceitualmente:

```
spring    -> D1, D2
boot      -> D1
security  -> D2
java      -> D1, D2
angular   -> D3
typescript -> D3
```

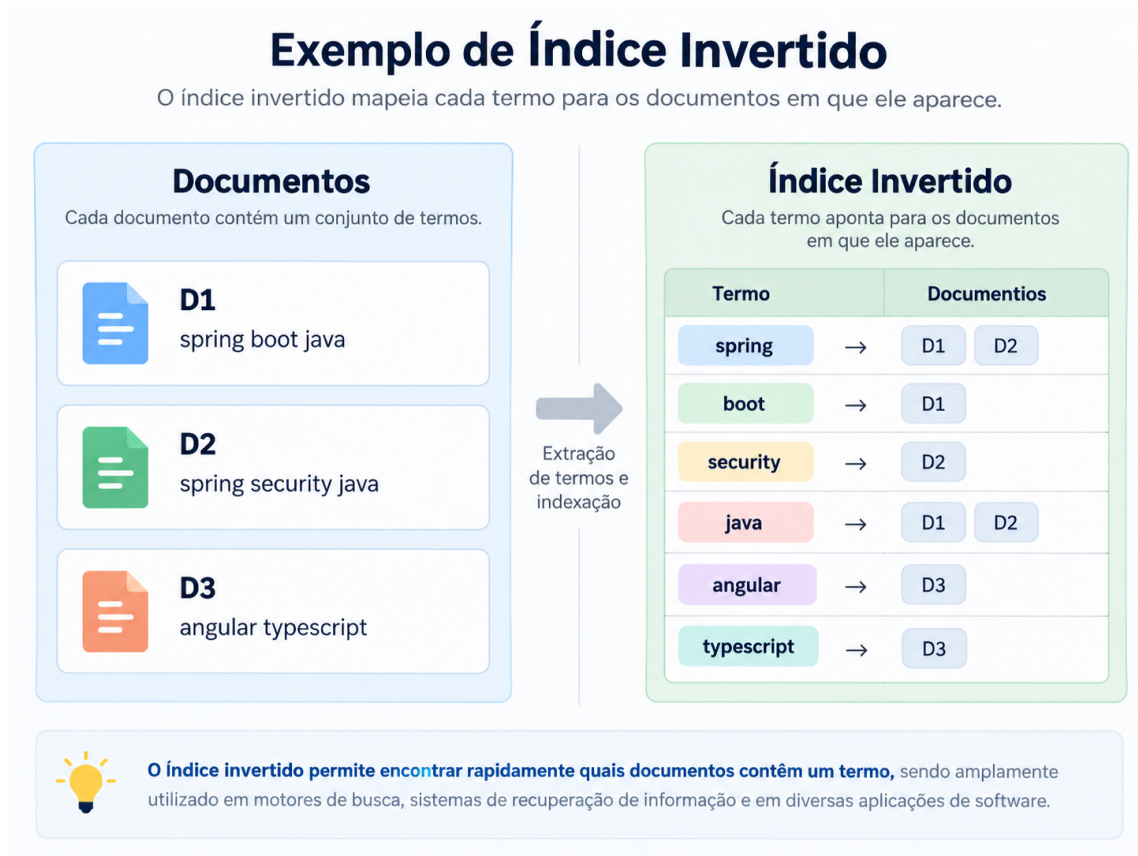


Figura 1: Exemplo visual de índice invertido

7.5 BM25

BM25 será o algoritmo inicial de ranking lexical.

Conceitualmente, o score considera:

- frequência do termo no documento;
- raridade do termo na coleção;
- tamanho do documento;
- parâmetros de normalização.

Ponto importante

Mesmo utilizando OpenSearch para a busca de produção, o projeto deverá possuir implementações experimentais próprias de algoritmos de RI para fins de estudo, comparação e avaliação.

8. Arquitetura geral

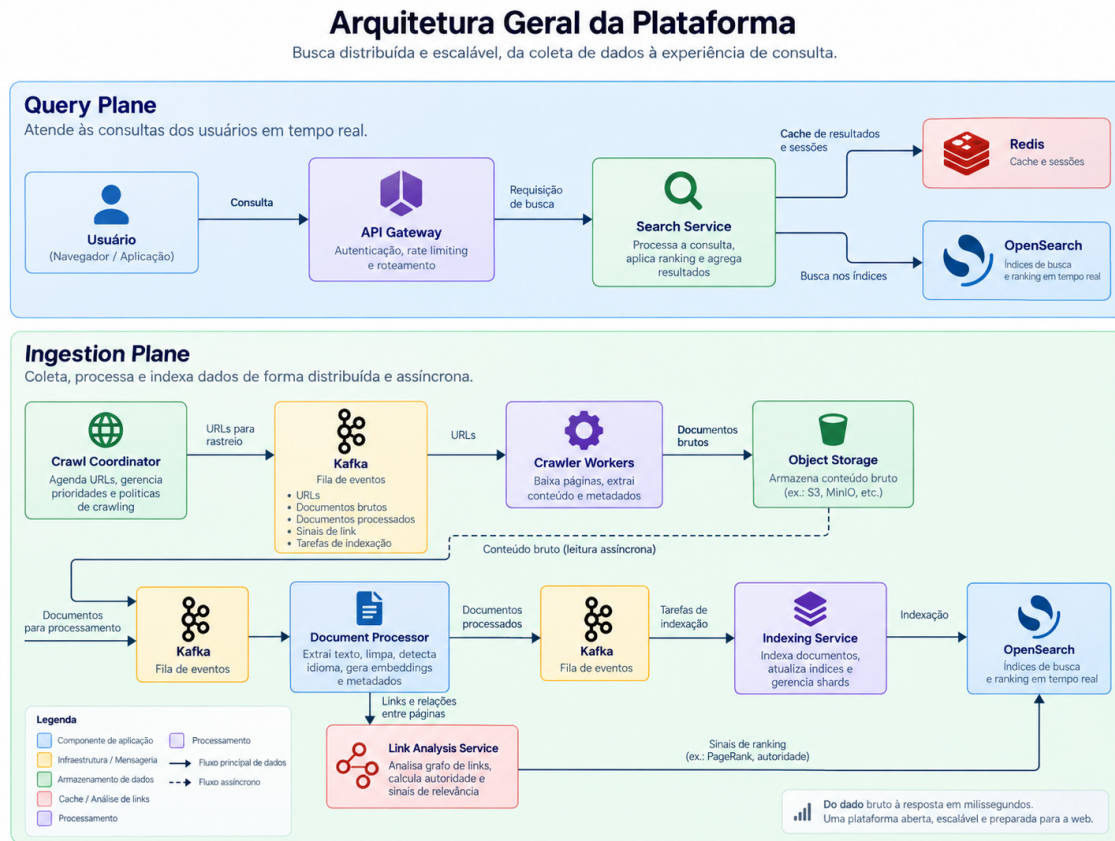
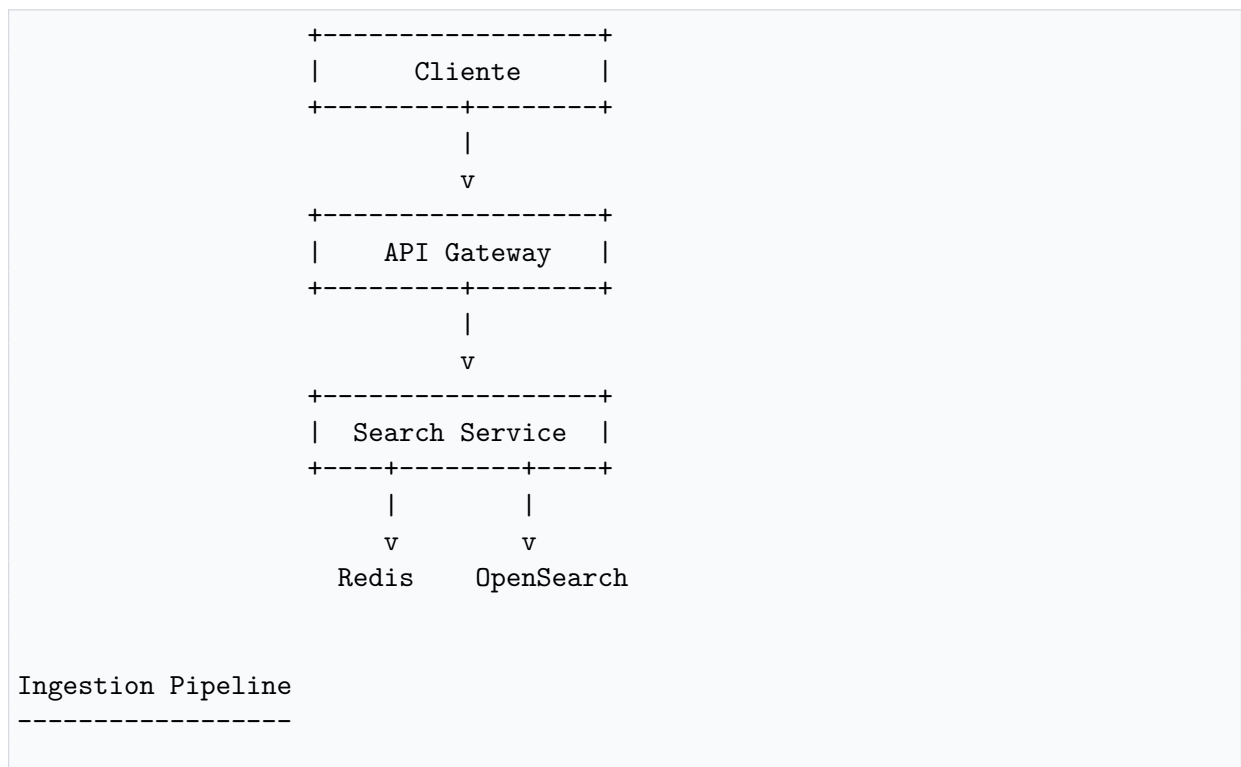
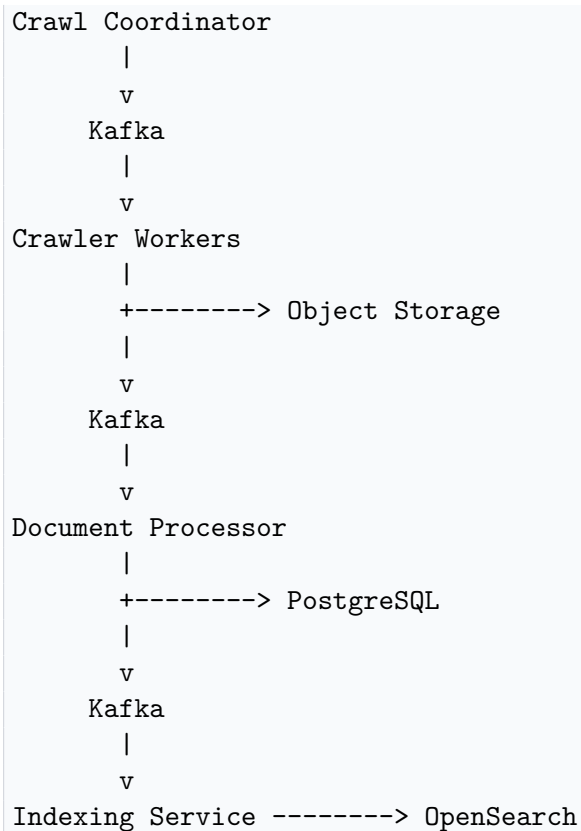


Figura 2: Arquitetura geral da plataforma

Arquitetura conceitual:





9. Microserviços

9.1 API Gateway

Responsabilidades:

- roteamento;
- autenticação administrativa;
- rate limiting;
- correlation ID;
- cabeçalhos comuns;
- observabilidade;
- proteção da API.

Tecnologia sugerida:

Spring Cloud Gateway

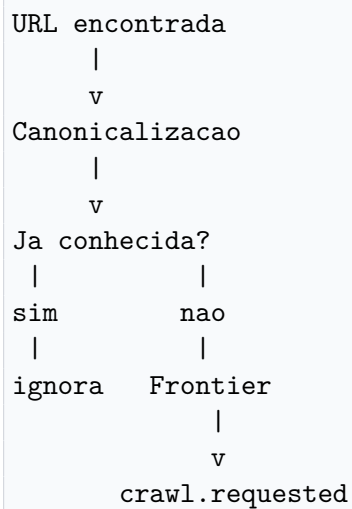
9.2 Crawl Coordinator

Responsável por coordenar o processo de crawling.

Responsabilidades:

- armazenar seeds;
- controlar o URL Frontier;
- priorizar URLs;
- aplicar políticas por domínio;
- identificar URLs já conhecidas;
- reagendar crawling;
- publicar tarefas de crawling.

Fluxo:



9.3 Crawler Worker

Responsável por efetivamente baixar páginas.

Responsabilidades:

- consumir tarefas de crawling;
- validar robots.txt;
- realizar HTTP request;
- limitar tamanho da resposta;
- tratar redirects;

- armazenar conteúdo bruto;
- calcular metadados básicos;
- emitir eventos.

A arquitetura deverá permitir algo como:

```
crawl-coordinator: 2 instancias
crawler-worker:    50 instancias
```

Como crawling é uma carga predominantemente I/O-bound, o uso de Virtual Threads deverá ser considerado e validado com benchmarks.

9.4 Document Processor

Pipeline conceitual:

```
HTML
|
v
Parser
|
v
Extracao
|
v
Normalizacao
|
v
Deteccao de idioma
|
v
Deduplicacao
|
v
Extracao de links
|
v
ParsedDocument
```

Bibliotecas possíveis:

- Jsoup;
- Apache Tika.

9.5 Indexing Service

Responsável exclusivamente por escrever no índice.

Principais responsabilidades:

- mappings;
- analyzers;
- bulk indexing;
- versionamento;
- exclusão;
- atualização;
- reindexação.

9.6 Search Service

Fluxo principal:

```
Query
|
v
Validacao
|
v
Normalizacao
|
v
Query Parsing
|
v
Recuperacao de candidatos
|
v
Ranking
|
v
Highlight / Snippet
|
v
Resposta
```

Decisão arquitetural

O Search Service deverá possuir poucas dependências síncronas e evitar depender de serviços do pipeline de ingestão durante o caminho crítico da pesquisa.

9.7 Link Analysis Service

Responsável por analisar o grafo formado pelos links.

Exemplo:

```
A -> B
A -> C
B -> C
C -> A
```

Primeiro algoritmo proposto:

PageRank

O resultado poderá ser incorporado como um sinal adicional de ranking.

9.8 Query Intelligence Service

Serviço futuro.

Possíveis responsabilidades:

- autocomplete;
- spelling correction;
- expansão de consultas;
- sinônimos;
- classificação de consultas;
- embeddings.

Ponto importante

Esse serviço não deve ser criado prematuramente. Inicialmente, funcionalidades simples podem permanecer dentro do Search Service e serem extraídas somente quando houver motivo arquitetural real.

10. Fluxo de crawling e indexação

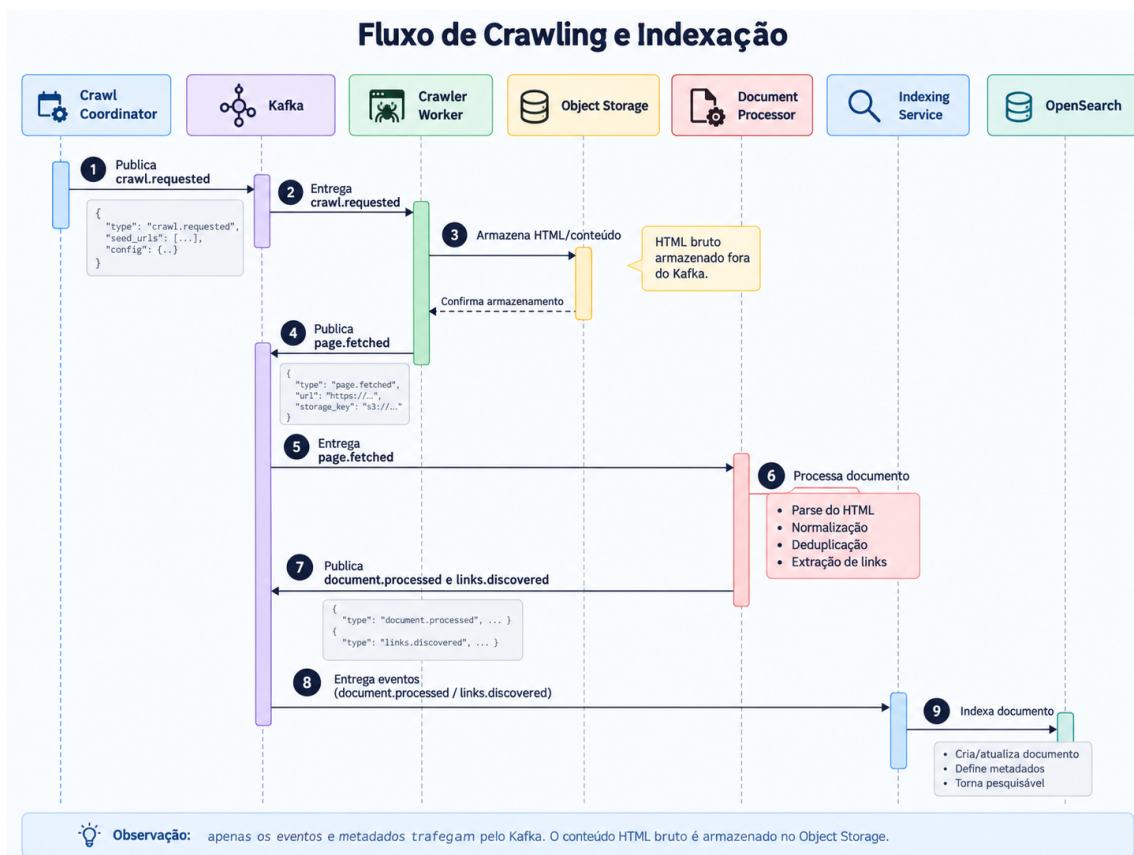


Figura 3: Fluxo de crawling e indexação

Boa prática

O HTML inteiro não deverá trafegar dentro de eventos Kafka. O evento deve transportar metadados e uma referência ao conteúdo armazenado no Object Storage.

Exemplo de evento:

```

PageFetched
|- eventId
|- documentId
|- url
|- storageKey
|- httpStatus
|- contentType
|- contentLength
|- fetchedAt
'- correlationId
  
```

11. Fluxo de pesquisa

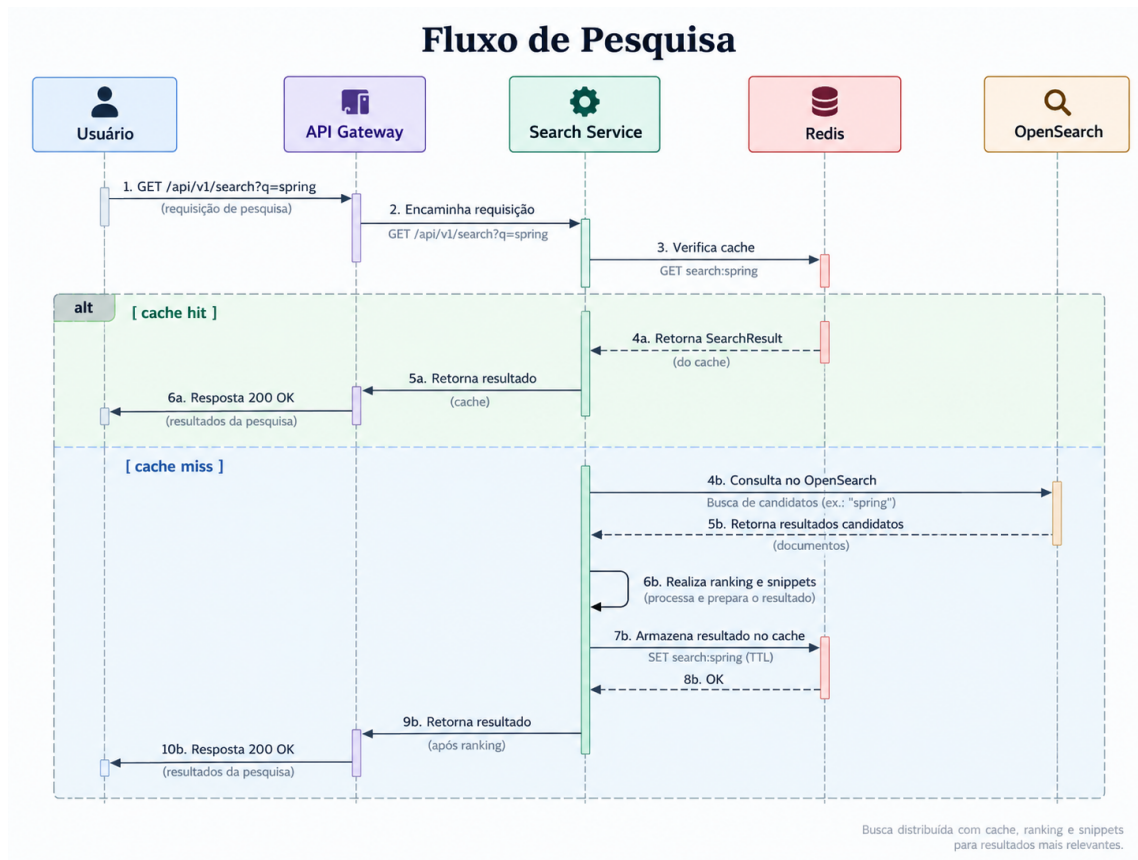


Figura 4: Fluxo de pesquisa

Fluxo conceitual:

```

Usuario
|
v
API Gateway
|
v
Search Service
|
+----> Redis
|      |
|      cache hit?
|      |
|      +----> resposta
|
+----> OpenSearch
|      |
|      v
|      candidatos
|      |
|      v
|      ranking + snippets
  
```

```
|  
v  
resposta
```

12. Estratégia de armazenamento

12.1 PostgreSQL

Utilizado para dados transacionais e metadados estruturados.

Exemplos:

```
crawl_seed  
crawl_policy  
domain  
url_metadata  
document_metadata  
crawl_execution
```

12.2 OpenSearch

Responsável pelos documentos pesquisáveis.

Exemplo conceitual:

```
{  
  "id": "...",  
  "url": "...",  
  "title": "...",  
  "description": "...",  
  "content": "...",  
  "language": "pt",  
  "pageRank": 0.32,  
  "indexedAt": "..."  
}
```

Será utilizado para:

- índice invertido;
- BM25;
- analyzers;
- stemming;
- highlighting;
- filtros;
- busca vetorial em uma etapa posterior.

12.3 Redis

Casos sugeridos:

- cache de buscas;
- rate limiting;
- URL Frontier;
- cache de robots.txt;
- locks temporários;
- deduplicação temporária.

12.4 Object Storage

Utilizado para:

- HTML bruto;
- snapshots;
- documentos grandes;
- datasets;
- exports;
- artefatos intermediários.

Em desenvolvimento, pode ser utilizado um storage S3-compatible local. Em produção, a implementação deverá poder ser substituída por um serviço cloud compatível através de abstração.

13. Mensageria e eventos

Kafka será utilizado para desacoplar o pipeline de ingestão.

Tópicos ou categorias de eventos sugeridos:

```
crawl.requested
page.fetched
page.fetch-failed
document.processed
document.rejected
links.discovered
document.index-requested
document.indexed
document.removed
ranking.updated
search.executed
```

Todo evento deverá possuir metadados mínimos:

```
eventId
eventType
eventVersion
occurredAt
correlationId
```

13.1 Garantia de processamento

A arquitetura deverá partir da premissa de:

```
at-least-once delivery
```

Consequentemente, consumidores devem ser idempotentes.

13.2 Schema Registry

Sugestão:

```
Apache Avro
+
Apicurio Registry
```

Isso permitirá versionamento e validação dos contratos de eventos.

14. Ranking e Recuperação de Informação

Uma evolução possível para a função de score:

$$Score = w_1 \cdot BM25 + w_2 \cdot PageRank + w_3 \cdot Freshness + w_4 \cdot FieldBoost + w_5 \cdot SemanticScore \quad (1)$$

Exemplo conceitual:

```
score =
  0.65 * lexicalScore
+ 0.15 * pageRank
+ 0.10 * freshness
+ 0.10 * semanticScore
```

Ponto importante

Os pesos acima são apenas ilustrativos. Devem ser tratados como hiperparâmetros e ajustados através de experimentação e métricas de relevância.

14.1 Ranking inicial

Primeira fase:

```
BM25
+
field boosting
```

Exemplo:

```
title^3
description^2
content^1
```

14.2 Segunda evolução

Adicionar PageRank.

14.3 Terceira evolução

Adicionar freshness para consultas onde recência seja importante.

14.4 Quarta evolução

Adicionar busca semântica:

```
Query
|
v
Embedding
|
v
Vector Search
```

14.5 Quinta evolução

Busca híbrida:

```
BM25 candidates
+
Vector candidates
|
v
Fusion
|
v
Reranking
```

15. Diagramas de classes

15.1 Crawl Coordinator

Classes principais:

```
CrawlSeed
|- UUID id
|- URI url
|- Instant createdAt
'- boolean active

CrawlTask
|- UUID id
|- URI url
|- int priority
|- Instant scheduledAt
'- CrawlStatus status

CrawlPolicy
|- String domain
|- Duration delay
|- int maxConcurrency
'- int maxDepth

UrlCanonicalizer
'- canonicalize(URI): URI

FrontierService
|- enqueue(URI)
|- next(): CrawlTask
'- reschedule(CrawlTask)

CrawlScheduler
'- schedule()

CrawlEventPublisher
'- publish(CrawlRequested)
```

15.2 Crawler Worker

15.3 Document Processor

15.4 Search Service

Diagrama de Classes — Search Service

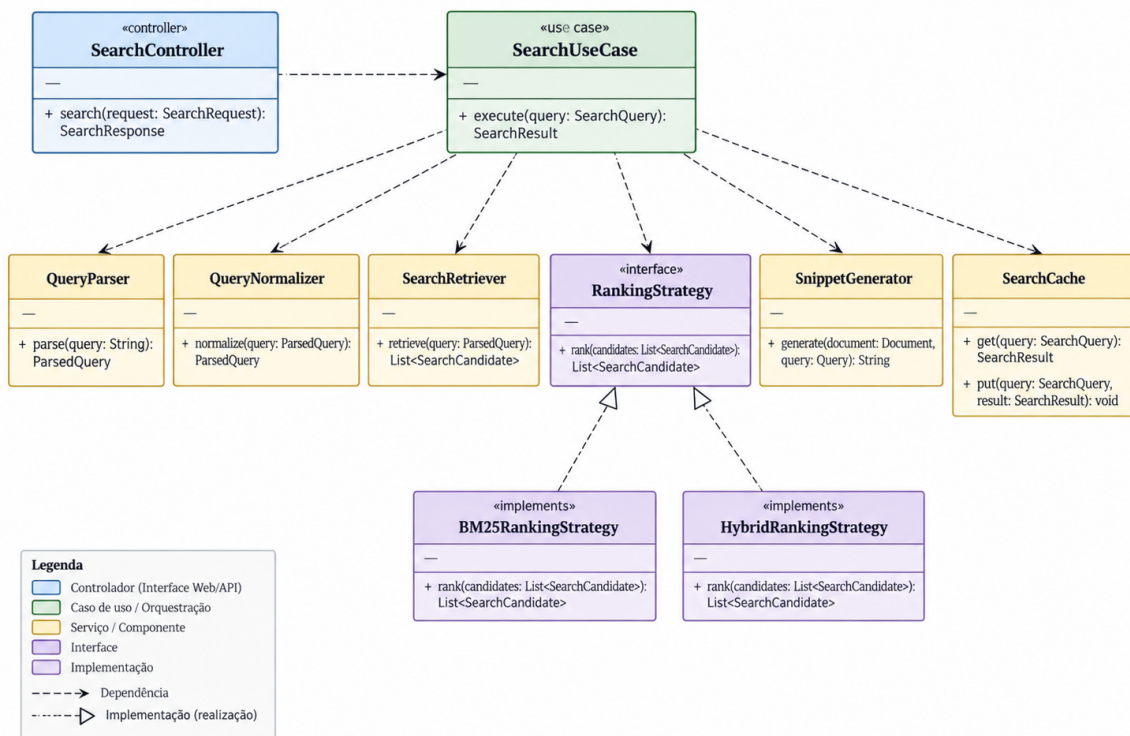


Figura 5: Diagrama de classes do Search Service

16. Arquitetura interna dos serviços

Evitar depender exclusivamente de uma estrutura baseada em:

```

controller
service
repository
entity
  
```

Para serviços mais complexos, preferir organização baseada em domínio e portas/adapters:

```

search
|-- domain
|   |-- model
|   |-- service
|   '-- port
|
|-- application
|   |-- usecase
|   |-- command
|   '-- query
|
  
```

```
|-- infrastructure
|   |-- opensearch
|   |-- kafka
|   |-- redis
|   '-- persistence
|
'-- interfaces
    |-- rest
    '-- messaging
```

Boa prática

O domínio não deverá depender diretamente de Kafka, PostgreSQL, Redis, OpenSearch ou do próprio Spring quando a separação arquitetural justificar essa abstração.

17. APIs principais

17.1 Search

```
GET /api/v1/search
```

Parâmetros:

```
q
page
size
language
```

Exemplo:

```
GET /api/v1/search?q=spring+boot&page=0&size=20
```

Resposta conceitual:

```
{
  "query": "spring boot",
  "tookMs": 42,
  "total": 12490,
  "results": [
    {
      "title": "Spring Boot",
      "url": "https://spring.io/projects/spring-boot",
      "snippet": "...",
      "score": 12.42
    }
  ]
}
```

17.2 Autocomplete

```
GET /api/v1/suggestions?q=spr
```

17.3 Administração de crawling

```
POST /api/v1/admin/crawl/seeds
```

17.4 Consulta de execução

```
GET /api/v1/admin/crawl/jobs/{id}
```

17.5 Recrawling

```
POST /api/v1/admin/documents/{id}/recrawl
```

17.6 Reindexação

```
POST /api/v1/admin/indexes/reindex
```

18. Escalabilidade e Kubernetes

Kubernetes será tratado como arquitetura-alvo.

Evolução recomendada:

```
Fase inicial
  |
  v
Docker Compose

Fase intermediaria
  |
  v
kind / k3d

Fase escalavel
  |
  v
Kubernetes
```

18.1 Serviços stateless

Devem preferencialmente ser Deployments:

- API Gateway;
- Crawler Worker;
- Document Processor;
- Indexing Service;
- Search Service.

18.2 Escalabilidade independente

Exemplo:

```
Search Service
replicas: 4 -> 15

Crawler Worker
replicas: 10 -> 100

Document Processor
replicas: 4 -> 30
```

18.3 HPA

Para o Search Service, podem ser consideradas métricas como:

- CPU;
- memória;
- latência;
- requests por segundo.

Para consumidores Kafka, uma métrica mais representativa pode ser o **consumer lag**.

18.4 KEDA

Como evolução, KEDA poderá ser utilizado para escalar consumidores com base em eventos e backlog.

```
Kafka lag aumenta
      |
      v
replicas aumentam
```

18.5 Service Discovery

Em Kubernetes, o próprio mecanismo de Services e DNS do cluster deverá ser utilizado.

Decisão arquitetural

Quando toda a arquitetura estiver executando em Kubernetes, não utilizar Eureka apenas para replicar uma função de descoberta de serviços que o próprio cluster já oferece.

18.6 Configuração

Utilizar inicialmente:

- ConfigMap;
- Secret.

Posteriormente, integrar um Secret Manager externo quando necessário.

19. Resiliência e consistência

A arquitetura deverá assumir que falhas parciais são normais.

Exemplos:

- Kafka temporariamente indisponível;
- OpenSearch respondendo lentamente;
- um crawler encerrado inesperadamente;
- um evento processado mais de uma vez;
- páginas retornando HTTP 500;
- URLs que nunca respondem.

19.1 Retry

Aplicar:

```
retry
+
exponential backoff
+
jitter
```

19.2 Circuit Breaker

Utilizar apenas em chamadas remotas síncronas onde fizer sentido.

Sugestão:

```
Resilience4j
```

19.3 Dead Letter Queue

Exemplo:

```
document.processed.dlq
```

19.4 Transactional Outbox

Quando um serviço precisar realizar simultaneamente:

```
salvar no PostgreSQL  
+  
publicar evento no Kafka
```

deverá ser avaliado o padrão Transactional Outbox.

Boa prática

Evitar transações distribuídas entre microserviços. Preferir consistência eventual, idempotência, outbox e processamento orientado a eventos.

20. Segurança

20.1 SSRF

O crawler não deve poder acessar livremente redes internas.

Devem existir regras para bloquear:

- localhost;
- loopback;
- redes privadas;
- link-local;
- endpoints de metadata de cloud;
- outros endereços considerados internos ao ambiente.

Também deverá existir proteção contra DNS rebinding.

20.2 Restrições HTTP

Definir:

- connect timeout;
- read timeout;
- tamanho máximo da resposta;
- máximo de redirects;
- tipos de conteúdo permitidos.

20.3 API administrativa

Endpoints sob:

```
/admin/**
```

deverão exigir autenticação.

Sugestão:

```
Keycloak  
+  
OAuth2 / OIDC
```

20.4 API pública

A API de busca poderá permanecer anônima, protegida por rate limiting e outras políticas de abuso.

21. Observabilidade

Todos os requests e eventos relevantes deverão carregar identificadores como:

```
traceId  
spanId  
correlationId
```

21.1 Métricas do Search Service

```
search_requests_total  
search_latency  
search_errors_total  
cache_hit_ratio  
opensearch_latency
```

21.2 Métricas do Crawler

```
pages_fetched_total  
pages_failed_total  
crawl_duration  
bytes_downloaded  
robots_blocked_total
```

21.3 Métricas Kafka

```
consumer_lag  
events_processed  
events_failed
```

21.4 Métricas do Indexer

```
documents_indexed_total  
indexing_latency  
bulk_size  
index_errors_total
```

21.5 Ferramentas

- Spring Boot Actuator;
- Micrometer;
- OpenTelemetry;
- Prometheus;
- Grafana;
- Loki;
- Tempo.

22. Testes e qualidade

22.1 Testes unitários

Ferramentas:

- JUnit;
- AssertJ;

- Mockito.

Componentes prioritários:

- UrlCanonicalizer;
- QueryParser;
- TextNormalizer;
- RankingStrategy;
- DuplicateDetector.

22.2 Testes de integração

Utilizar Testcontainers para infraestrutura real durante testes:

- PostgreSQL;
- Kafka;
- Redis;
- OpenSearch.

22.3 Testes arquiteturais

Sugestão:

```
ArchUnit
```

Exemplo de regra:

```
domain nao depende de infrastructure
```

22.4 Contratos

- OpenAPI para REST;
- Avro para eventos.

22.5 Performance

Ferramentas possíveis:

- k6;
- Gatling.

Cenários:

- throughput de pesquisa;
- throughput de crawling;
- throughput de indexação;
- recuperação após backlog elevado no Kafka.

23. Avaliação de Recuperação de Informação

Criar um dataset de relevância contendo:

Query	Documento	Relevância
spring boot	spring.io	3
spring boot	oracle.com	1
spring boot	angular.io	0

23.1 Precision@K

Dos primeiros K resultados, quantos realmente são relevantes?

23.2 Recall@K

Dos documentos relevantes existentes, quantos foram recuperados?

23.3 MRR

Avalia a posição do primeiro resultado relevante.

23.4 nDCG@K

Considera:

- grau de relevância;
- posição do resultado.

23.5 Experimentos

A plataforma deverá permitir comparar, por exemplo:

```
Experimento A
BM25

Experimento B
BM25 + PageRank

Experimento C
BM25 + PageRank + Field Boost

Experimento D
Hybrid Search
```

24. Estratégia de índices

Evitar acoplamento dos serviços com nomes físicos de índices.

Em vez de:

```
documents_v17
```

utilizar aliases:

```
documents-read
documents-write
```

Estratégia de reindexação:

```
documents_v1
documents_v2

documents-read -> documents_v1
documents-write -> documents_v2

apos concluir:

documents-read -> documents_v2
documents-write -> documents_v2
```

Boa prática

A aplicação deve conhecer aliases estáveis. A infraestrutura de indexação controla qual versão física do índice está ativa.

25. Organização do repositório

Um monorepo inicialmente simplificará o desenvolvimento.

```
search-engine/  
|  
|-- services/  
|   |-- api-gateway/  
|   |-- crawl-coordinator/  
|   |-- crawler-worker/  
|   |-- document-processor/  
|   |-- indexing-service/  
|   |-- search-service/  
|   '-- link-analysis-service/  
|  
|-- contracts/  
|   |-- kafka/  
|   '-- openapi/  
|  
|-- platform/  
|   |-- docker/  
|   |-- kubernetes/  
|   |-- helm/  
|   '-- observability/  
|  
|-- docs/  
|   |-- architecture/  
|   |-- diagrams/  
|   '-- adr/  
|  
|-- datasets/  
|   '-- relevance/  
|  
'-- README.md
```

26. Compartilhamento de código

Evitar um grande módulo genérico:

```
common
```

compartilhado indiscriminadamente por todos os serviços.

Podem existir módulos pequenos e bem definidos, como:

```
platform-observability  
event-contracts
```

Entidades de domínio não devem ser compartilhadas apenas por conveniência.

Por exemplo:

```
Document
```

do Document Processor não precisa ser a mesma classe Java de:

```
SearchDocument
```

do Indexing Service.

27. Estratégia database-per-service

Cada serviço deverá ser proprietário de seus próprios dados.

Conceitualmente:

```
crawl-coordinator
  |
  v
crawl-db

document-processor
  |
  v
document-db

indexing-service
  |
  v
OpenSearch
```

Na primeira versão, bancos PostgreSQL podem compartilhar a mesma instância física, desde que possuam separação lógica de:

- database;
- usuário;
- migrations;
- ownership.

Ponto importante

Dois microserviços não devem alterar diretamente as mesmas tabelas.

28. Arquitetura de deploy

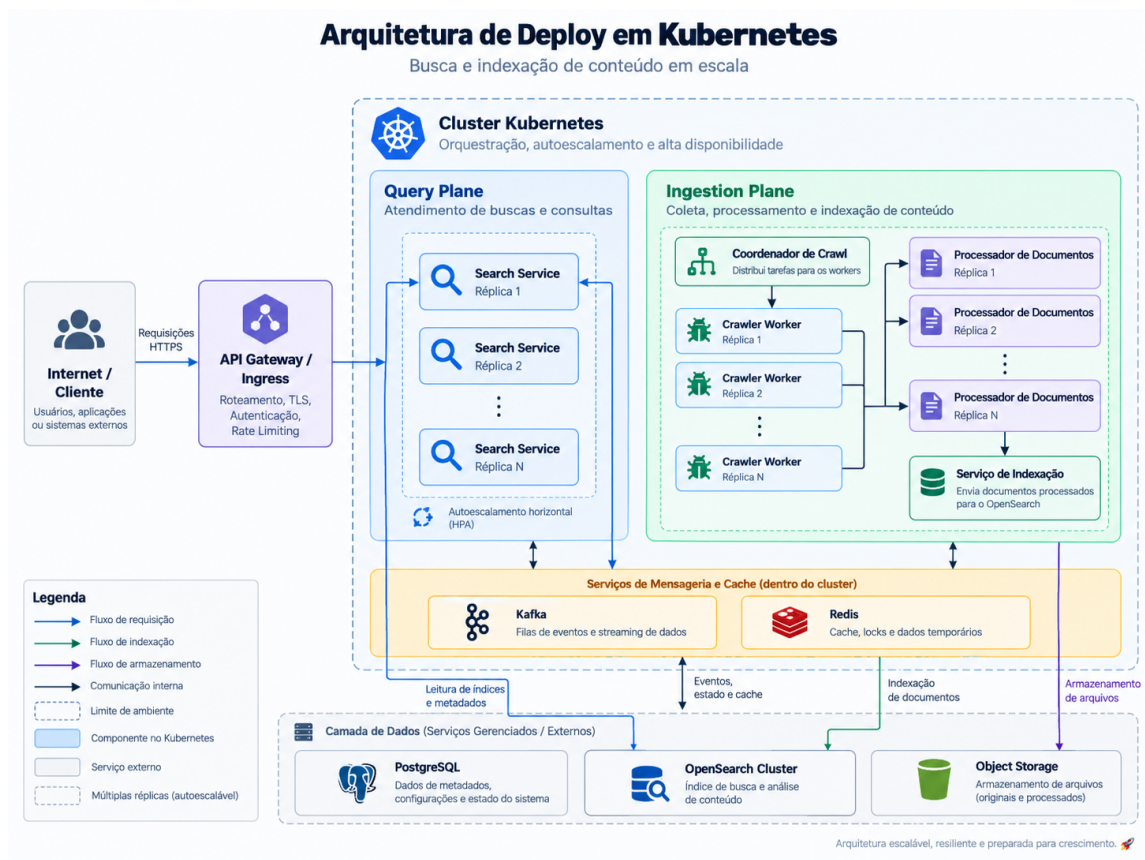


Figura 6: Arquitetura de deploy em Kubernetes

29. Roadmap de desenvolvimento

29.1 Fase 1 — Recuperação de Informação básica

Criar:

- Search Service;
- Indexing Service;
- OpenSearch;
- PostgreSQL.

Implementar:

- documentos manuais;
- tokenização;
- BM25;

- busca;
- ranking;
- snippets.

29.2 Fase 2 — Crawler

Adicionar:

- Crawl Coordinator;
- Crawler Worker;
- Document Processor.

29.3 Fase 3 — Arquitetura orientada a eventos

Adicionar:

- Kafka;
- Object Storage.

Desacoplar crawler, processor e indexer.

29.4 Fase 4 — Crawling distribuído

Adicionar:

- URL Frontier;
- Redis;
- robots.txt;
- politeness;
- deduplicação;
- priorização;
- recrawling.

29.5 Fase 5 — Qualidade de ranking

Adicionar:

- PageRank;
- field boosting;
- freshness;
- datasets de relevância;
- Precision@K;
- Recall@K;
- MRR;
- nDCG.

29.6 Fase 6 — Kubernetes

Implementar:

- Deployments;
- Services;
- ConfigMaps;
- Secrets;
- HPA;
- health checks;
- resource limits.

29.7 Fase 7 — Observabilidade

Adicionar:

- OpenTelemetry;
- Prometheus;
- Grafana;
- Loki;
- Tempo.

29.8 Fase 8 — Busca semântica

Adicionar:

- Embedding Service;
- Vector Search;
- Hybrid Search;
- Reranking.

30. O que implementar manualmente e o que utilizar pronto

30.1 Vale implementar ou experimentar

Para aprendizado de Recuperação de Informação:

- tokenização;
- normalização;
- stemming;
- TF-IDF;
- modelo vetorial;
- similaridade de cosseno;
- BM25 experimental;
- query processing;
- combinação de scores;
- PageRank;
- métricas de avaliação;
- ranking.

Pode existir um módulo específico:

```
ri-core
```

contendo implementações didáticas e experimentais.

30.2 Não vale reinventar inicialmente

Evitar implementar do zero:

- cluster de índice distribuído;
- replicação;
- consenso distribuído;
- broker de mensagens;
- container orchestration;
- distributed tracing;
- storage distribuído.

Utilizar ferramentas maduras para essas responsabilidades.

31. Decisões arquiteturais principais

ADR-001 — Microserviços

A plataforma será dividida por capacidades de negócio e características de escalabilidade, não simplesmente por entidades.

ADR-002 — Kafka

O pipeline de ingestão será orientado a eventos.

ADR-003 — OpenSearch

OpenSearch será o índice principal de documentos pesquisáveis.

ADR-004 — PostgreSQL

PostgreSQL armazenará metadados e dados transacionais.

ADR-005 — Redis

Redis será utilizado para cache e informações temporárias.

ADR-006 — Object Storage

Conteúdo bruto não trafegará integralmente dentro de mensagens Kafka.

ADR-007 — Kubernetes

Kubernetes será a arquitetura-alvo, mas não será requisito para desenvolvimento local.

ADR-008 — Service Discovery

Em Kubernetes, o service discovery nativo deverá ser utilizado no lugar de Eureka.

ADR-009 — Consistência eventual

Não utilizar transações distribuídas entre microserviços.

ADR-010 — Database per service

Cada serviço será proprietário de seus dados.

ADR-011 — Query Plane isolado

O pipeline de pesquisa não dependerá do pipeline de crawling para responder consultas.

ADR-012 — RI própria + infraestrutura madura

Algoritmos relevantes academicamente serão implementados e avaliados, enquanto infraestrutura distribuída será delegada a soluções maduras.

32. Stack final recomendada

```
Java 25
Spring Boot 4
Spring Cloud

PostgreSQL
OpenSearch
Redis
Apache Kafka
S3-compatible Object Storage

Docker
Docker Compose
Kubernetes

OpenTelemetry
Prometheus
Grafana
Loki
```

```
Tempo

JUnit
AssertJ
Mockito
Testcontainers
ArchUnit
k6 ou Gatling

Maven
Flyway
GitHub Actions
Argo CD
```

33. Princípio central da arquitetura

O projeto deverá preservar quatro características:

Crawling escalável
+
Indexação assíncrona
+
Pesquisa de baixa latência
+
Ranking experimental

A arquitetura deverá poder crescer de:

```
1 crawler
1 search-service
1 OpenSearch
```

para:

```
centenas de crawler workers
dezenas de consumidores Kafka
cluster OpenSearch
múltiplas instâncias de Search Service
autoscaling Kubernetes
```

sem exigir uma reescrita completa da plataforma.

Ponto importante

A evolução por etapas, mantendo contratos claros e componentes desacoplados, deve orientar todas as decisões futuras do projeto.